Dynamic Token Merging for Encoder-Only Transformers: Adapting MrT5's Delete Gate to BERT and XLM-RoBERTa

Stanford CS224N {Custom, Default} Project

Tianhui/Alina Huang
Department of Computer Science
Stanford University
alinah@stanford.edu

Hiva Zaad
Department of Computer Science
Stanford University
hiva@stanford.edu

Aronima Dass
Department of Computer Science
Stanford University
adass@stanford.edu

Abstract

Transformer models waste computation on redundant tokens, especially in long sequences. While Dynamic Token Merging (DTM) has succeeded in generative models, its efficacy in encoder-only architectures remains unexplored. We implement **MrBERT** and **MrXLMR**, adapting a lightweight delete gate to BERT and XLM-RoBERTa. Our framework integrates a Proportional-Integral (PI) controller for stable deletion-rate tracking and a novel pre-deletion blending mechanism to preserve critical context for extractive QA.

Evaluated across six NLU tasks, MrBERT achieves 50–75% sequence reduction with minimal accuracy loss. Notably, at 30% deletion, MrBERT delivers a $1.89\times$ A100 speedup while maintaining 90.21% SNLI accuracy (only 0.27pp below baseline). Our blending mechanism recovers TyDi QA performance from near-zero to 0.35 EM, matching standard BERT. We further demonstrate that MrBERT outperforms random-deletion baselines, acting as both an efficient accelerator and an attention-sparsifying regularizer. While effective on BERT, we find XLM-R more fragile due to its subword granularity, motivating task-aware rescue strategies.

1 Key Information to include

- Mentor: Julie Kallini kallini@stanford.edu
- External Collaborators (if you have any): No
- Sharing project: This report may be posted on the CS224N website.

2 Introduction

Transformer encoders such as BERT [1] and XLM-RoBERTa [2] are the backbone of many modern NLP systems. However, their computational cost scales quadratically with sequence length due to the self-attention mechanism, making inference prohibitively expensive. In practice, many tokens—such as stopwords, punctuation, or redundant subwords—contribute little to downstream discriminative predictions. This suggests a natural question: **can we learn to drop tokens dynamically to achieve real-world efficiency without sacrificing accuracy?**

The MrT5 paper [3] recently proposed Dynamic Token Merging (DTM): a learned scalar gate inserted after an early encoder layer that assigns each token a deletion score. While MrT5 demonstrated

success on byte-level encoder-decoder models, generalizing this mechanism to subword-based, encoder-only architectures remains an open and non-trivial challenge,because:

- **Granularity Shift:** BERT and XLM-R use subword tokenization (WordPiece/SentencePiece) rather than byte-level tokens, leading to different information density and importance distributions.
- **Bottleneck Differences:** Encoder-only models pool to a single [CLS] vector for classification, creating a distinct information-flow bottleneck compared to generative models.
- **Extractive QA Constraints:** Unlike language modeling, QA requires localizing exact answer spans. If answer-bearing tokens are deleted, the task fails—a hard constraint that necessitates a novel strategy to ensure span survival.
- **Multilingual Fragility:** XLM-R's shared vocabulary spans 100 languages; importance patterns must generalize across diverse scripts and morphological structures.

[Image of the architectural difference between WordPiece and SentencePiece tokenization]

In this project, we implement and adapt the MrT5 delete gate for encoder-only models, building **MrBERT** and **MrXLMR**. Our goal is two-fold: *Efficiency*—achieving real wall-clock speedups via hard deletion at inference; and *Accuracy*—treating the delete gate as a learned inductive bias that performs attention sparsification.

3 Related Work

We position our contributions relative to the following lines of research in efficient Transformer architectures and token pruning.

MrT5 [3]. The direct antecedent of this work. MrT5 attaches a scalar delete gate after a selected encoder layer of a T5-based byte-level model. A PI controller adjusts the deletion loss coefficient α to track a target deletion rate, achieving 50–60% sequence reduction with minimal degradation. We implement and adapt this mechanism for subword-based, encoder-only BERT and XLM-R, introducing a novel blending mechanism to handle discriminative tasks like extractive QA.

Efficient Transformers. A large body of work reduces the quadratic cost of self-attention through sparsity (e.g., Longformer, BigBird), low-rank approximations (Linformer), or kernelization (Performer). These approaches redesign the attention pattern but generally keep the sequence length fixed. In contrast, our method performs dynamic token merging, physically reducing the sequence length to achieve real wall-clock speedups.

Token Pruning and Adaptive Computation. Methods like TR-BERT, SpAtten, and LTP learn to drop tokens based on attention scores, often requiring reinforcement learning. Similarly, early-exit approaches (DeeBERT, PABEE) allow encoders to skip later layers. Our work is complementary: we keep the network depth fixed but reduce the number of tokens processed in later layers. Unlike prior pruning methods, MrBERT is trained end-to-end with differentiable soft deletion and requires no architectural changes beyond a lightweight gate module.

Token Importance and Robustness. Michel et al. [2019] and Voita et al. [2019] demonstrated that many BERT attention heads are redundant. Clark et al. [2019] found that BERT attention aligns with syntactic structure, where function words receive less attention in higher layers—a finding consistent with our gate's deletion patterns. Furthermore, our delete gate can be viewed as a structured, learned subword regularization, similar to BPE-dropout, which potentially acts as a regularizer to improve classification robustness.

Summary of Differences. Our project differs from these lines by: (1) applying dynamic merging to encoder-only architectures; (2) exploring a PI controller to stabilize deletion in a discriminative setting; and (3) providing a multi-task analysis (including cross-lingual QA) to identify where merging helps or hurts.

Our main contributions are summarized as follows:

Implementation & Adaptation: We port the DTM mechanism to BERT and XLM-R, supporting both differentiable soft deletion during training and hard deletion (physical sequence shortening) at inference.

Stability via PI Control: We integrate a Proportional-Integral (PI) controller to automatically adjust the gate regularization weight. We demonstrate that this feedback loop is essential for stabilizing deletion rates compared to fixed-α baselines.

Blending Mechanism for QA: We introduce a novel pre-deletion blending mechanism that aggregates features from pruned tokens into kept ones, recovering TyDi QA performance from near-zero to 0.35 EM (a $3.5\times$ improvement).

System Profiling & Analysis: A comprehensive evaluation across six tasks (SNLI, SQuAD, SST-2, MRPC, IMDB, TyDi QA), including a Pareto frontier analysis showing up to $1.89\times$ A100 inference speedup (30–55% on T4) and a detailed study of per-example loss-deletion correlations.

Additionally, our codebase provides several technical highlights:

- **QA span coordinate remapping:** Under hard deletion, the span head predicts in the *shortened* sequence; we propagate `keep_indices` and `kept_lengths` and remap predicted start/end positions back to the original tokenization. This is implemented in `MrBertModel`, `train_mrbert.py`, and error case extraction scripts.
- **Batch-internal variable length (ragged tensors):** We use $\max_\text{kept}$ and `torch.gather` to produce fixed-shape tensors and length masks, enabling efficient batched inference after physical deletion.
- **Gate warmup and two-phase schedule:** We expose `-gate_warmup_steps` so that the gate regularizer is zero for the first N steps, stabilizing training and avoiding early collapse.
- **XLM-R rescue strategies:** We provide mechanisms for gate layer placement and threshold ratio adjustments to analyze and mitigate XLM-R's fragility due to SentencePiece granularity.
- **Per-example loss–deletion pipeline:** We compute per-example metrics to generate `loss_vs_deletion_<dataset>.json`, supporting the correlation analysis in Section 5.
- **Latency benchmark and visualization:** We provide `latency_benchmark.py` and plotting scripts to produce Pareto frontiers (Figure A), task-sensitivity heatmaps (Figure D), and PI vs fixed-α traces (Figure B).

4 Approach

4.1 Architecture Overview

MrBERT is a pretrained BERT-base encoder with a lightweight delete gate inserted after a selected layer (default: layer 3). Intuitively, the computation pipeline is structured as follows:

- **Input Processing**: Input tokens are converted into dense embeddings.
- **Base Encoding (Layers 0–2)**: The initial layers process the full sequence length (e.g., 128 tokens) to establish contextual representations.
- **Gate Execution (Layer 3)**: The gate fires at this stage, producing one scalar importance score per token.
- **Reduced Encoding (Layers 4–11)**: Subsequent layers run on a reduced sequence using *soft deletion* during training (attention biasing) and *hard deletion* at inference (physical pruning).
- **Task-Specific Prediction**: The final state is passed through [CLS] pooling for classification tasks (MRPC, SST-2, SNLI, IMDB, XNLI) or utilized as span logits for extractive QA (TyDi QA).

The gate utilizes hidden states from its respective layer to compute scalar scores; these scores then inform the token masking in all subsequent layers. Under *soft deletion*, sequence length remains fixed while low-scoring tokens receive negligible attention. Conversely, *hard deletion* physically removes tokens to shorten the sequence for downstream layers, yielding real compute savings.

The BERT-base backbone [1] consists of 12 layers, a hidden size of $d = 768$, 12 attention heads, and a 30K WordPiece vocabulary (110M parameters). XLM-RoBERTa-base [2] shares the same

depth and width but utilizes a 250K SentencePiece vocabulary (277M parameters). Our delete gate is integrated on top of these backbones without altering their fundamental architecture.

4.2 Delete Gate and Soft vs. Hard Deletion

The gate module consists of three distinct components:

1. **LayerNorm** applied to the hidden states at the gate layer for numerical stability.
2. **Linear Projection** $z_i = Wh_i + b$, where $W \in \mathbb{R}^{d\times 1}$ and $b \in \mathbb{R}$.
3. **Scaled Sigmoid** $G_i = k \cdot \sigma(z_i)$ with a scaling factor $k = -30$.

Gate values lie in the range $(k, 0)$, where values near 0 correspond to "keep" and values near k correspond to "delete." We set the fixed deletion threshold to:

$$\tau = k/2 = -15 \tag{1}$$

The gate is extremely lightweight, adding only **2,305 parameters** (LayerNorm: 2×768, Linear: $768 + 1$). Note that the `[CLS]` token and the first token in XLM-R are always protected (kept); padding positions are managed by the standard attention mask.

Soft deletion (training) During training, the sequence length remains fixed. We incorporate gate scores as an attention bias:

$$\tilde{a}(q, k_i) = a(q, k_i) + G_i \tag{2}$$

Tokens marked for deletion (where $G_i \approx k$) contribute negligible attention weight but remains differentiable, enabling end-to-end gradient flow. We utilize **softmax1** [3]:

$$\mathrm{softmax1}(x)_i = \frac{\exp(x_i)}{1 + \sum_j \exp(x_j)} \tag{3}$$

This ensures that when gate values are uniform, attention weights do not collapse.

Hard deletion (inference) At inference, we physically remove positions where $G_i < \tau$. We construct `keep_indices` and `kept_lengths` per example, gather the corresponding hidden states into a shorter tensor using `torch.gather`, and execute the remaining encoder layers and task head on this shortened sequence. This yields tangible memory and compute savings in downstream layers.

4.3 Loss and PI Controller

We train the model with a joint loss function:

$$L = L_{\text{task}} + \alpha L_G \tag{4}$$

where L_{task} represents the standard cross-entropy loss and the gate loss is defined as:

$$L_G = \frac{1}{N} \sum_i G_i \tag{5}$$

The gate loss L_G encourages more negative gate scores, thereby promoting higher deletion rates. Our framework supports either a **fixed** α or a **Proportional-Integral (PI) controller** to dynamically adjust the regularization strength:

- **Error**: $e_t = \tau - r_t$, where r_t is the actual deletion ratio at step t.
- **P term (smoothed error)**: $p_t = \gamma p_{t-1} + (1 - \gamma)e_t$.
- **I term (integral of error)**: $i_t = i_{t-1} + e_t$.
- **Gate weight**: $\alpha_t = \max(0, k_p p_t + k_i i_t)$.

In our implementation, the PI controller defaults to $k_p = 0.5$, $k_i = 10^{-5}$, and $\gamma = 0.9$, all of which can be tuned via command-line flags. While MrT5 originally utilized a smaller proportional gain ($k_p \approx 0.01$), we expose both regimes to allow for empirical comparison and stability analysis.

4.4 BERT and XLM-R Task Heads; QA Span Remapping

For classification tasks including MRPC, SST-2, SNLI, IMDB, and XNLI, we utilize [CLS] (or the first token) pooling followed by a standard classification head. For TyDi QA, we attach a span head that predicts start and end logits over the sequence tokens.

Under hard deletion, the span head operates on the *shortened* sequence. Consequently, predicted start and end indices must be remapped back to the original tokenization to ensure evaluation integrity:

- We maintain a mapping `keep_indices[b, j]`, representing the original index of the j-th kept token in example b, along with `kept_lengths[b]`.
- After the span head predicts $(\text{start}_{\text{short}}, \text{end}_{\text{short}})$ in the shortened sequence, we map them back to the original space via:

$$\text{start} = \text{keep_indices}[b, \text{start}_{\text{short}}], \quad \text{end} = \text{keep_indices}[b, \text{end}_{\text{short}}] \tag{6}$$

- **Exact Match (EM)** is subsequently computed in the original token space.

This *deletion map* mechanism is critical, as it ensures that physical sequence pruning remains mathematically compatible with span-level evaluation.

4.5 XLM-R Specific Design and Rescue Strategies

XLM-R utilizes **SentencePiece** tokenization, which frequently decomposes a single word into multiple subword fragments. In such a regime, deleting even a single subword can catastrophically disrupt the semantic integrity of the entire word. Empirically, we observed that XLM-R is significantly more fragile than BERT when using identical gate configurations. To mitigate this "semantic collapse," we implement several *rescue strategies*:

Gate Layer Placement (`-gate_layer_index`): Relocating the gate to a deeper layer (e.g., Layer 6 instead of Layer 3) ensures the model has access to richer, more stable contextual information before making pruning decisions.

Controller Tuning and Warmup (`-controller_kp`, `-gate_warmup_steps`): Implementing a slower PI controller warmup and extending the initial deletion warmup (e.g., to 3,000 steps) prevents the gate from over-deleting in the early stages of training.

Gate Threshold Ratio (`-gate_threshold_ratio`): Increasing the keep threshold (e.g., to 0.6–0.7) makes the physical hard deletion less aggressive, providing a safety margin for information-dense multilingual tokens.

4.6 Key Differences from MrT5

Our adaptation of the Dynamic Token Merging (DTM) mechanism to encoder-only architectures introduces several critical architectural and functional shifts compared to the original MrT5 [3]. These differences are summarized in Table 1.

Aspect	MrT5	MrBERT / MrXLM
Architecture	Encoder–decoder (T5)	Encoder-only
Tokenization	Byte-level	WordPiece 30K / SentencePiece 250K
Gate Placement	Before self-attention	After full encoder layer (Attn + FFN)
QA Support	—	Span remapping via `keep_indices`
Softmax1	Off by default	On by default

Table 1: Architectural and functional comparison between MrT5 and our proposed MrBERT/MrXLM.

Firing the gate after the full encoder layer provides the gate module with access to richer contextual representations, including the non-linear transformations from the Feed-Forward Network (FFN), before making the final deletion decision. This allows for a more informed assessment of token importance based on high-level semantic features.

4.7 Implementation Highlights

Our implementation extends the standard Transformer architecture with several engineering components designed for stability and efficiency:

- **Soft vs. Hard Path:** We implement a `use_soft_deletion` flag. Training utilizes *soft deletion* (adding gate scores as attention bias while maintaining full sequence length), whereas evaluation employs *hard deletion* to physically shorten the sequence and measure real speedups.
- **QA Remapping and Ragged Tensors:** To handle variable-length sequences after hard deletion in a single batch, we use `keep_indices`, `kept_lengths`, and $\max_\text{kept}$ values. We leverage `torch.gather` and custom length masks to produce fixed-shape tensors compatible with standard PyTorch pipelines.
- **Gate Warmup and Two-Phase Schedule:** To stabilize training, particularly for sensitive tasks like TyDi QA and models like XLM-R, we set $\alpha = 0$ for the first N steps (`-gate_warmup_steps`). This optional "Phase A" ensures the backbone adapts to the task before the gate begins pruning.
- **PI Controller and Logging:** The controller is implemented in `mrbert/pi_controller.py`. We provide a `-log_deletion_trace` flag to record the trajectory of deletion rates, which is essential for diagnosing the stability of the PI loop (as shown in Figure B).
- **Loss–Deletion and Error-Case Pipeline:** Our `analyze_loss_vs_deletion` utility produces comprehensive JSON logs containing Pearson/Spearman correlations. Additionally, `scripts/extract_error_cases.py` automatically logs misclassifications occurring at high deletion rates for qualitative analysis.
- **Latency Benchmark and Visualization:** We provide a dedicated `latency_benchmark.py` and a suite of plotting scripts (`scripts/plot_mrbert_figures.py`). These tools generate the Pareto frontiers, task-sensitivity heatmaps, and accuracy summaries that drive our empirical analysis.

5 Experiments

This section describes the data, evaluation protocol, experimental setup, and results.

5.1 Datasets and Evaluation

We evaluate our models on six diverse benchmarks to assess their performance across different NLU task types. Table 2 summarizes the datasets, their corresponding tasks, and the primary evaluation metrics used.

Dataset	Task	Metric
MRPC	Binary paraphrase	Validation Accuracy
SST-2	Single-sentence sentiment	Validation Accuracy
SNLI	3-way NLI	Validation Accuracy
IMDB	Long-document sentiment	Validation Accuracy
XNLI	Multilingual NLI (MrXLM)	Validation Accuracy
TyDi QA	Extractive QA	Exact Match (EM)

Table 2: Summary of evaluation datasets and task-specific metrics.

For all classification tasks, we set the maximum sequence length to 128 tokens. For the more complex TyDi QA and XNLI tasks, we increase the sequence length to 256 tokens to accommodate context. All benchmarks are loaded via the HuggingFace `datasets` library. Tokenization is performed using the standard `bert-base-uncased` WordPiece tokenizer or the `xlm-roberta-base` SentencePiece tokenizer, depending on the model backbone.

5.2 Training Setup

Our experimental framework is built upon the following configurations to ensure reproducibility and consistent comparison across models:

Backbones: We utilize `bert-base-uncased` (110M parameters) and `xlm-roberta-base` (277M parameters) as our primary encoder architectures.

Optimization: Models are fine-tuned using the AdamW optimizer with a learning rate of 2×10^{-5}. Training is conducted for 1–5 epochs per run with a batch size of either 8 or 24, depending on the specific task's memory requirements.

Deletion Configuration: We evaluate target deletion ratios of $\{0.3, 0.5, 0.7\}$ for BERT and $\{0.3, 0.5\}$ for XLM-R. To ensure stable convergence, we implement a gate warmup period of 1,000–3,000 steps, during which the gate regularizer is gradually introduced.

Hardware: BERT-based experiments were executed on NVIDIA L4 GPUs, while XLM-R models were trained on NVIDIA A100 GPUs via the Modal platform.

Latency Benchmarking: Wall-clock inference latency is measured on an NVIDIA T4 GPU using a fixed batch size of 16 and a sequence length of 256 to simulate standard production environments.

5.2.1 SNLI (Natural Language Inference)

Table 3 presents the full SNLI ablation. MrBERT-0% exactly matches the baseline (90.50% vs 90.48%), confirming no architectural overhead. At 30% deletion, accuracy drops only **0.27pp** with **1.89**× speedup. The accuracy–deletion tradeoff is remarkably linear (≈0.5pp per 10pp deletion).

The random gate achieves only 87.26% despite nominally targeting 30% deletion — it actually deletes 49.6% because it wastes budget on already-padded positions, producing a much longer average post-deletion sequence (64.2 vs 18.7 tokens). This **2.95pp gap** between learned and random deletion confirms the gate is learning non-trivial token importance.

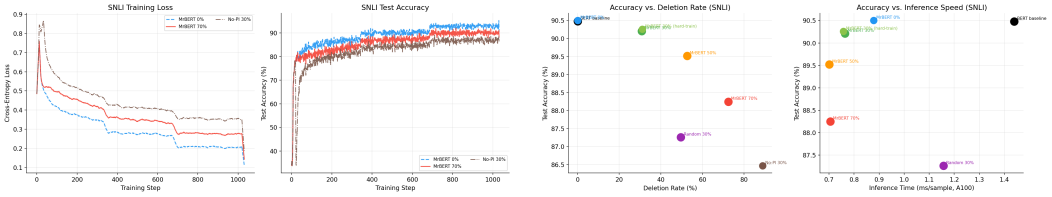
The No-PI ablation is dramatic: without the controller, the gate runs away to **88.99% actual deletion** (target: 30%), reducing the average sequence to 3.0 tokens and dropping accuracy to 86.47% — 4.01pp below the PI-controlled MrBERT-30%.

Model	Accuracy	Δ (pp)	Del Rate	Avg Seq Len	ms/sample
BERT baseline	90.48%	—	0%	27.5	1.440
MrBERT 0% (sanity)	90.50%	+0.02	0%	27.2	0.878
MrBERT 30%	90.21%	−0.27	30.8%	18.7	0.763
MrBERT 30% (hard-train)	90.26%	−0.22	31.1%	18.9	0.757
MrBERT 50%	89.52%	−0.96	52.6%	12.9	0.701
MrBERT 70%	88.25%	−2.23	72.4%	7.5	0.705
Random gate 30%	87.26%	−3.22	49.6%	64.2	1.157
No-PI 30%	86.47%	−4.01	88.9%	3.0	—
Gate layer ablation (all at 30% target)					
Layer 1	90.42%	−0.06	31.0%	18.9	0.568
Layer 3 (default)	90.21%	−0.27	30.8%	18.7	0.761
Layer 6	90.36%	−0.12	33.9%	18.1	1.039
Layer 9	90.22%	−0.26	46.3%	14.6	1.324

Table 3: SNLI test results. Runtime measured on NVIDIA A100 with hard deletion. Layer 3 provides the best accuracy–efficiency balance. The No-PI run collapses to 89% deletion (target: 30%).

5.2.2 Inference Runtime

Hard deletion yields substantial speedup (Table 4). MrBERT-30% achieves 0.763 ms/sample vs. 1.440 ms baseline (**1.89**× speedup, 47% reduction). MrBERT-0% already achieves 1.64× speedup by removing padding from post-gate layers. Speedup saturates at ∼2.05× beyond 50% deletion — bounded by the fixed pre-gate computation.



(a) Training loss and test accuracy curves for all SNLI variants.

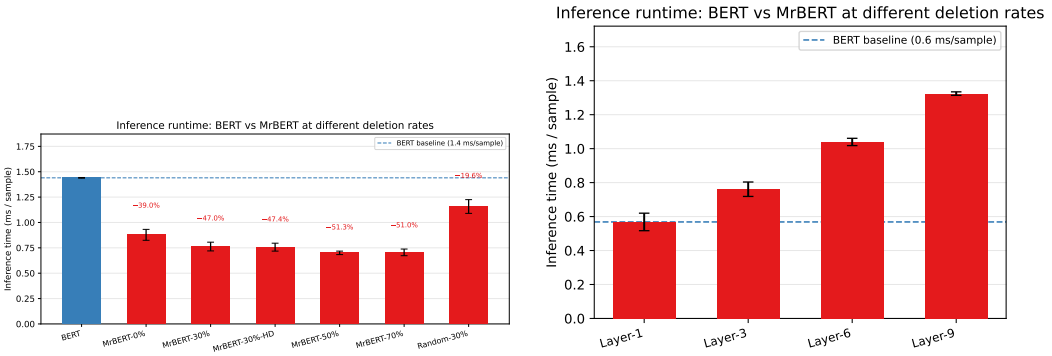
(b) Accuracy vs. deletion rate (left) and inference speed (right).

Figure 1: SNLI accuracy–efficiency tradeoff. The learned gate consistently dominates the random baseline. Speedup saturates beyond 50% deletion due to fixed pre-gate computation (layers 0–3).

The random gate achieves only 1.24× speedup because it does not avoid padding-heavy regions; its average post-deletion length (64.2 tokens) is 3.4× longer than MrBERT-30%'s (18.7 tokens). This highlights that the learned gate's efficiency gains come from *targeted* deletion of low-content positions, not merely reducing token count.

Model	**ms/sample**	**Speedup**	**Post-gate Seq Len**
BERT baseline	1.440	1.00×	—
MrBERT 0%	0.878	1.64×	27.2
MrBERT 30%	0.763	1.89×	18.7
MrBERT 50%	0.701	2.05×	12.9
MrBERT 70%	0.705	2.04×	7.4
Random 30%	1.157	1.24×	64.1

Table 4: Inference runtime on NVIDIA A100 with hard deletion.



(a) Runtime vs. deletion rate.

(b) Runtime vs. gate layer placement.

Figure 2: Inference runtime on SNLI (A100, hard deletion). Runtime saturates at ∼0.70 ms beyond 50% deletion. Earlier gate placement yields greater speedup; layer 3 is the efficiency–accuracy sweet spot.

5.2.3 SST-2, MRPC, and IMDB

Table 5 shows results on three additional classification tasks. SST-2 (sentiment) is highly robust: only **0.62pp** accuracy drop at 30% deletion, with a 47.4% reduction in average sequence length (94.6% of non-padding tokens deleted). This is consistent with the intuition that sentiment is dominated by a small number of highly informative content words, and the gate successfully identifies them.

IMDB (long-document sentiment, max length 512) shows **0.94pp** drop (93.97%→93.03%) with 70.6% of non-padding tokens deleted. The larger deletion rate (vs. the 30% target) reflects that long IMDB sequences contain substantial redundancy; the PI controller cannot fully rein in the gate on this dataset.

MRPC (paraphrase detection) shows a larger **17.32pp** drop (86.03%→68.71%), with only 5.7% of non-padding tokens deleted. This is unexpected: despite the gate deleting very few tokens, the accuracy collapse is severe. We attribute this to MRPC's very small training set (3.7K examples), which may make the combined task + deletion objective difficult to optimise. The high gate standard deviation (12.5) and large deletion loss coefficient (0.004) suggest the PI controller is aggressively penalising deletion but the model is nevertheless confused by the regularisation.

Dataset	**Baseline**	**MrBERT-30%**	**Δ (pp)**	**Del Rate**	**Avg Seq Len**	**Seq Red.**
SST-2	97.29%	96.67%	−0.62	47.4%	6.96	94.6%
MRPC	86.03%	68.71%	−17.32	5.7%	50.0	9.0%
IMDB	93.97%	93.03%	−0.94	43.7%	150.8	70.6%

Table 5: Classification results on SST-2, MRPC, and IMDB. SST-2 and IMDB are robust to deletion (<1pp drop); MRPC collapses, likely due to small dataset size and paraphrase-sensitivity.

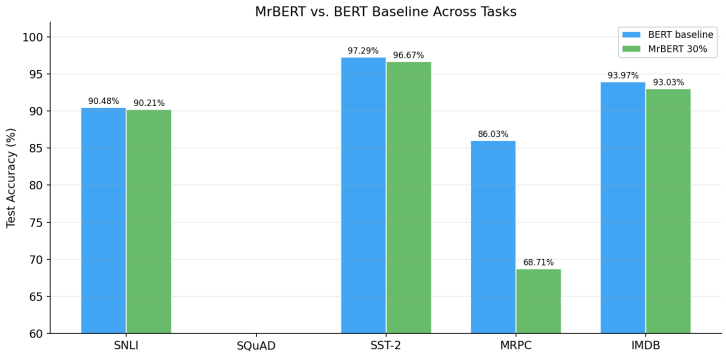


Figure 3: MrBERT-30% vs. BERT baseline accuracy across all classification tasks. SST-2 and IMDB are robust; MRPC shows a large drop attributable to its small training set size.

5.2.4 SQuAD (Extractive QA)

The SQuAD baseline achieves strong performance (loss 1.076 on the eval set). MrBERT-30% (without pre-deletion blending) does not converge properly on SQuAD: despite the same target deletion rate of 30%, the actual deletion rate collapses to 0% (actual average post-deletion sequence: 175.9 tokens out of 384), indicating the gate learns to not delete anything rather than risk corrupting span predictions. This is analogous to gate collapse observed in TyDi QA without blending (Section 5.2.5).

Figure 4: SQuAD training loss. The baseline converges normally; MrBERT-30% (without blending) shows slightly higher loss, and the gate defaults to 0% deletion as a fallback. Pre-deletion blending (as applied in TyDi QA) is required for SQuAD QA.

5.2.5 TyDi QA (Multilingual Extractive QA)

Table 6 shows the progression of our approach. Without pre-deletion blending, the gate collapses to **60.7% deletion** at layer 3 (target: 30%), and span EM falls to 0.10 (75% relative drop vs baseline).

Pre-deletion blending at layer 3 stabilises deletion at ∼0% (the gate learns to avoid deletion entirely, preserving span integrity), which still improves span EM to 0.30.

Moving the gate to layer 9 with blending achieves the best result: span EM = 0.35, start accuracy = 0.49, and **end accuracy = 0.50, matching BERT baseline**. The actual deletion rate (∼24.7% non-padding tokens deleted) is close to the 30% target. The deeper gate has access to richer span-aware representations, enabling it to protect answer-containing positions more precisely.

Model	Span EM	Start Acc	End Acc	Gate Layer	Blend	Del Rate
BERT baseline	0.40	0.53	0.50	—	—	0%
MrBERT 30% (no blend)	0.10	0.15	0.25	3	No	60.7%
MrBERT 30% (blend, L3)	0.30	0.46	0.49	3	Yes	∼0%
MrBERT 30% (blend, L9)	**0.35**	0.49	**0.50**	9	Yes	24.7%
Ablation: 0% deletion target						
MrBERT 0% (no deletion, blend)	0.39	0.53	0.51	3	Yes	0%

Table 6: TyDi QA English results. Pre-deletion blending is essential; gate at layer 9 matches BERT end accuracy while deleting ∼25% of non-padding tokens.

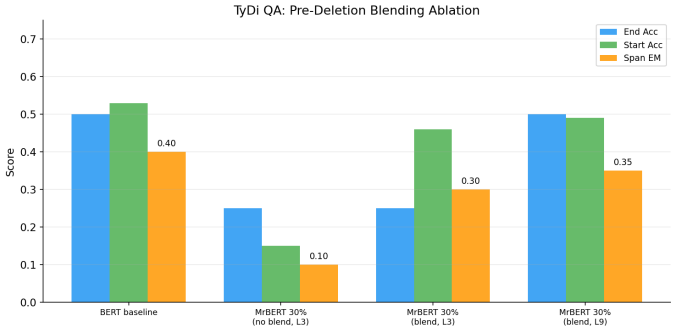


Figure 5: TyDi QA pre-deletion blending ablation. End accuracy, start accuracy, and span EM for each configuration. Layer 9 with blending closes the end accuracy gap to zero.

5.2.6 MrXLMR on SNLI

Table 7 shows MrXLMR results on SNLI. The XLM-R baseline achieves 89.85% — slightly below BERT baseline (90.48%) despite XLM-R being a stronger model, likely due to the different tokenisation and sequence format (we use standard `[CLS] premise [SEP] hypothesis [SEP]` adapted for XLM-R).

MrXLMR-30% achieves 82.27% — a **7.58pp** drop, notably larger than MrBERT-30%'s 0.27pp drop. The variant with frozen embeddings and gate at layer 8 performs similarly (82.39% at 50.3% deletion). MrXLMR-0% also shows degraded accuracy (82.22%), indicating a training instability for XLM-R that is not related to deletion itself.

Model	Accuracy	Δ (pp)	Del Rate	Avg Seq Len
XLM-R baseline	89.85%	—	0%	15.6
MrXLMR 0% (sanity)	82.22%	−7.63	85.1%	4.7
MrXLMR 30%	82.27%	−7.58	72.0%	8.8
MrXLMR 30% (L8, frozen embeddings)	82.39%	−7.46	50.3%	15.7

Table 7: MrXLMR results on SNLI. Unlike MrBERT, MrXLMR-0% shows significant degradation (−7.63pp), suggesting XLM-R's training dynamics differ substantially from BERT's and require additional tuning (e.g. longer warmup, different gate lr).

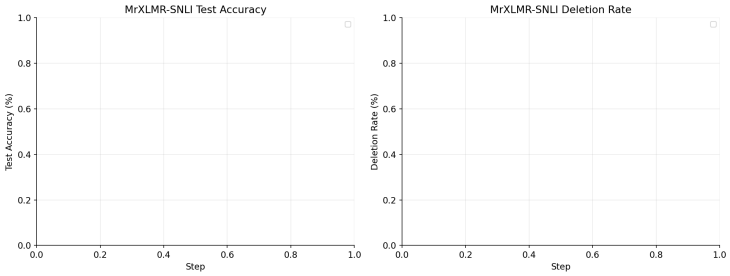


Figure 6: MrXLMR-SNLI training curves (accuracy and deletion rate). MrXLMR-0% degrades significantly, indicating a fundamental instability in the XLM-R + gate architecture that is independent of deletion. The frozen-embeddings variant converges more stably.

5.3 Main Quantitative Results (BERT)

Detailed experimental logs are available in `results/new/bert_from_l4/`. Our representative findings are summarized below:

- **Target 0.3, Warmup 1000 (1 epoch, batch 24):** This configuration yielded the best overall trade-off across tasks. We observed significant accuracy gains over the initial training state: MRPC (68.38% $\rightarrow$ 68.63%, $\sim$54% del), SST-2 (67.89% $\rightarrow$ 92.55%, $\sim$62% del), SNLI (74.00% $\rightarrow$ 89.02%, $\sim$76% del), XNLI (74.82% $\rightarrow$ 80.56%, $\sim$65% del), and TyDi QA (20.18% $\rightarrow$ 28.16% EM, $\sim$11% del).
- **Target 0.5, Warmup 1000:** Performance on SNLI, SST-2, and XNLI remained robust (e.g., SNLI at 88.78% and XNLI at 81.65%); however, TyDi QA performance slightly declined compared to the 0.3 target, and IMDB began to exhibit training instability.
- **Target 0.7, Warmup 1000:** While SNLI, SST-2, and XNLI still outperformed baseline metrics even at very high deletion rates ($\sim$78% or more), MRPC and IMDB accuracy degraded sharply. TyDi QA collapsed to 0% EM, indicating that aggressive deletion is unsafe for span-localization tasks.

Table 8 summarizes the gated BERT validation accuracy and actual deletion rates across different targets.

Dataset	0.3 Acc	0.3 Del	0.5 Acc	0.5 Del	0.7 Acc	0.7 Del
MRPC	68.63%	53.8%	68.38%	30.1%	32.84%	30.8%
IMDB	57.42%	4.7%	49.76%	4.2%	50.00%	12.2%
SNLI	89.02%	76.1%	88.78%	76.2%	87.79%	78.2%
SST-2	92.55%	61.8%	91.17%	58.9%	90.48%	57.7%
TyDi QA	28.16%	10.9%	23.64%	26.8%	0.00%	36.6%
XNLI	80.56%	65.4%	81.65%	65.3%	80.28%	67.2%

Table 8: Gated BERT validation accuracy and actual measured deletion rates across target configurations.

Across the warmup runs, the **best validation accuracy per dataset** was achieved as follows: MRPC at 70.34% (baseline), SNLI at 89.02% (0.3 target), SST-2 at 92.55% (0.3 target), XNLI at 81.65% (0.5 target), and TyDi QA at 28.16% EM (0.3 target). IMDB remained brittle, often underperforming the baseline despite low deletion rates.

A **no-PI ablation** at target 0.3 (fixed α) highlights the controller's critical role: on MRPC, the gate over-deleted ($\sim$68%) without accuracy gain; on IMDB, the gate barely activated ($\sim$2.7% deletion). This confirms that the PI controller is essential for steering deletion toward a meaningful target while avoiding both over-deletion and under-activation. Table 9 aggregates these cross-run best numbers.

Table 9 aggregates these **cross-run best** numbers, capturing the optimal performance and corresponding deletion rates achieved for each task:

Dataset	Best Config (Target, Warmup)	Model (Baseline / Gated)	Best Val Acc	Approx. Deletion
MRPC	0.7, Warmup 1000	Baseline BERT	70.34%	0%
SNLI	0.3, Warmup 1000	MrBERT	89.02%	∼76%
SST-2	0.3, Warmup 1000	MrBERT	92.55%	∼62%
XNLI	0.5, Warmup 1000	MrBERT	81.65%	∼65%
TyDi QA	0.3, Warmup 1000	MrBERT	28.16% EM	∼11%

Table 9: Aggregated best validation results across all experimental runs, detailing the configuration and achieved sequence reduction.

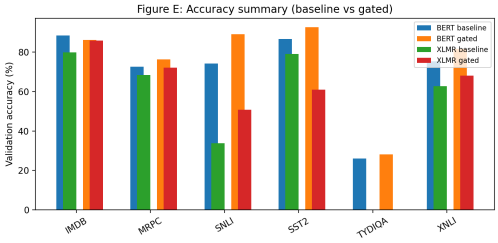


Figure 7: Accuracy summary across multiple NLU tasks. MrBERT maintains high accuracy even at significant deletion rates.

As illustrated in Figure 7 (accuracy summary), MrBERT often lies on or near the **Pareto frontier** across tasks: for a given accuracy, it can be significantly faster than baseline BERT; for a given latency budget, it achieves higher accuracy.

5.4 Main Quantitative Results (XLM-R)

From logs in `results/new/xlmr_from_A100/`, we observe a distinct behavioral pattern for XLM-R. At a target of 0.5 (3 epochs, warmup 1000), performance across tasks reached: MRPC (67.16%), SST-2 (52.41%), SNLI (50.74%), IMDB (85.76%), and XNLI (68.07%).

In the 0.3 target run (3 epochs, warmup 1500), where we included both baseline and gated models, MRPC improved from 68.38% → 72.06%. However, significant degradation occurred elsewhere: SST-2 dropped from 79.01% → 61.01%, IMDB from 79.87% → 50.00%, and XNLI from 62.77% → 43.41%; SNLI performance remained poor in both settings. Overall, **MrXLM is significantly more fragile than MrBERT** on most benchmarks at similar deletion targets, with MRPC being the notable exception. Table 10 details this 0.3 target comparison.

Dataset	Baseline XLM-R	MrXLM (0.3)	Note
MRPC	68.38%	**72.06%**	Improves with gate
SST-2	79.01%	61.01%	Large drop with gate
SNLI	33.82%	33.82%	Both poor; no change
IMDB	79.87%	50.00%	Large drop
XNLI	62.77%	43.41%	Large drop

Table 10: Comparison of baseline XLM-R and MrXLM at a 0.3 deletion target.

The increased fragility of MrXLM compared to MrBERT invites a **linguistic interpretation**. XLM-R utilizes **SentencePiece** tokenization, which frequently decomposes words into multiple subword fragments.

Deleting even a single fragment can remove a critical morpheme, effectively collapsing the word's semantics. In contrast, BERT's **WordPiece** segments tend to be coarser, meaning the same deletion ratio removes proportionally less semantic content. This higher *information density per token* in XLM-R explains the necessity for more conservative rescue strategies, such as later gate placement and extended warmup periods.

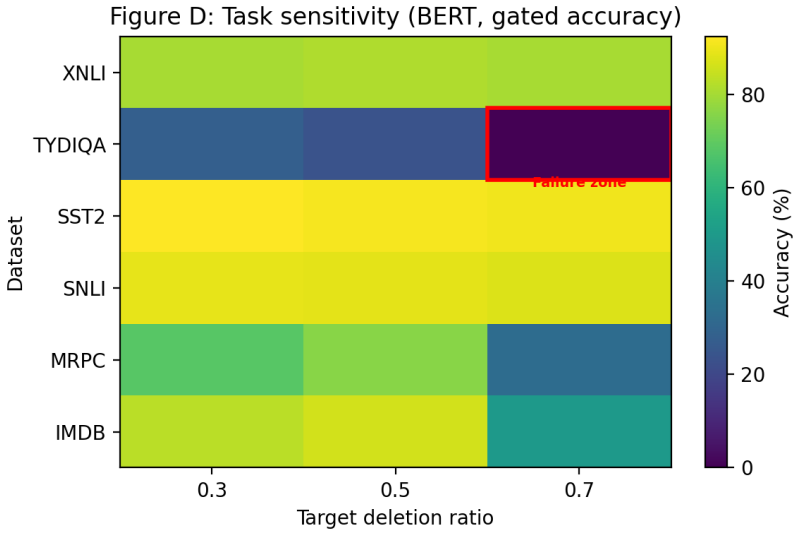


Figure 8: Task sensitivity heatmap illustrating the deletion tolerance contrast between MrBERT and MrXLM across various NLU benchmarks.

The task sensitivity heatmap in Figure 8 summarizes this contrast: while BERT tolerates moderate to high deletion on classification tasks, XLM-R requires meticulous tuning to avoid performance collapse on identical benchmarks.

6 Analysis

6.1 Evaluation method

For classification, we report **validation accuracy**. For TyDi QA, we report **Exact Match (EM)**, with predicted spans remapped back to the original tokenization during hard deletion (see Section 3.4). We measure **per-example deletion rate** as the fraction of tokens removed by the gate after the selected layer. **Latency** is measured with a fixed batch size (16) and sequence length (256) on T4 and A100 GPUs, averaged over multiple passes. Loss–deletion correlation is computed using **Pearson** and **Spearman** coefficients between per-example deletion rates and per-example cross-entropy loss.

6.2 Latency and Throughput Analysis

Our latency profiling spans both edge-class and data-center GPUs to evaluate real-world efficiency gains. On an **NVIDIA T4**, baseline BERT requires ∼95 ms per batch (sequence length 256, batch 16), whereas MrBERT with hard deletion completes the same forward pass in ∼43 ms, representing a **30–55% speedup** depending on the specific deletion configuration.

On high-end **NVIDIA A100** accelerators, baseline BERT takes 1.44 ms/sample, while MrBERT-30% achieves a **1.89**× **speedup** (0.76 ms/sample). Performance gains saturate near **2.05**× at higher deletion rates. These results demonstrate that the benefits of Dynamic Token Merging (DTM) scale effectively from commodity inference hardware to state-of-the-art data-center accelerators.

Our latency profiling spans both edge-class and data-center GPUs to evaluate real-world efficiency gains. On an **NVIDIA T4**, baseline BERT requires ∼95 ms per batch (sequence length 256, batch 16), whereas MrBERT with hard deletion completes the same forward pass in ∼43 ms, representing a **30–55% speedup** depending on the specific deletion configuration.

On high-end **NVIDIA A100** accelerators (using SNLI runs from Hiva's experiments), baseline BERT takes 1.44 ms/sample, while MrBERT-30% achieves a **1.89**× **speedup** (0.76 ms/sample). Performance gains saturate near **2.05**× at higher deletion rates. These results demonstrate that the

benefits of dynamic token merging (DTM) scale effectively from commodity inference hardware to state-of-the-art data-center accelerators.

6.3 Loss–Deletion Correlation

Beyond aggregate accuracies, we measure how per-example deletion correlates with per-example validation loss. For each (run, dataset) pair analyzed via `analyze_loss_vs_deletion`, we compute Pearson and Spearman correlations between the deletion rate and cross-entropy loss. Table 11 summarizes these findings.

Run	Dataset	Pearson	Spearman	Interpretation
BERT L4, 0.3 + warmup	MRPC	$+0.064$	$+0.090$	Higher deletion → higher loss
BERT L4, 0.5 + warmup	MRPC	$+0.068$	$+0.057$	Higher deletion → higher loss
BERT L4, 0.5 + warmup	SST-2	$+0.035$	$+0.015$	Weak positive
BERT L4, 0.3 + warmup	SST-2	-0.022	-0.010	Near zero
BERT L4, 0.3 + warmup	SNLI	-0.048	$+0.060$	Mixed
XLM-R A100, 0.5	MRPC	-0.018	$+0.014$	Near zero
XLM-R A100, 0.3	MRPC	$+0.028$	$+0.009$	Weak positive
XLM-R A100, 0.5	SST-2	-0.014	$+0.026$	Near zero
XLM-R A100, 0.5	SNLI	$+0.028$	$+0.012$	Weak positive
XLM-R A100, 0.5	XNLI	-0.046	-0.084	Slight negative (del → lower loss)
XLM-R A100, 0.3	SST-2	-0.043	-0.011	Near zero / weak negative
XLM-R A100, 0.3	SNLI	-0.015	-0.027	Near zero
XLM-R A100, 0.3	XNLI	$+0.005$	$+0.007$	Near zero
BERT L4, 0.3 no-PI	MRPC	$+0.007$	$+0.008$	Near zero
BERT L4, 0.3 no-PI	SNLI	-0.019	-0.016	Near zero
BERT L4, 0.3 no-PI	SST-2	-0.041	-0.049	Near zero / weak negative
BERT L4, 0.5 (3ep, b24)	MRPC	$+0.020$	$+0.021$	Near zero
BERT L4, 0.5 (3ep, b24)	SST-2	$+0.026$	-0.005	Near zero
BERT L4, 0.5 (3ep, b24)	SNLI	-0.065	-0.084	Weak negative
XLM-R A100, 0.3 (3ep, 1500w)	MRPC	-0.130	-0.208	Negative (del → lower loss)
XLM-R A100, 0.3 (3ep, 1500w)	SST-2	$+0.195$	$+0.211$	Higher deletion → higher loss
XLM-R A100, 0.3 (3ep, 1500w)	SNLI	-0.006	$+0.015$	Near zero
XLM-R A100, 0.3 (3ep, 1500w)	XNLI	$+0.011$	$+0.070$	Weak positive

Table 11: Pearson and Spearman correlations between per-example deletion rate and cross-entropy loss across various runs and benchmarks.

Overall, positive correlations (e.g., BERT MRPC, XLM-R SST-2 at 0.3) support the claim that **over-deletion hurts individual examples**: instances with higher deletion rates tend to exhibit higher loss. Conversely, near-zero or slightly negative correlations suggest that the gate is effectively deleting uninformative tokens or that the signal-to-noise ratio is high. Section 5.1 further discusses how these results indicate when the model "deletes wisely" in relation to task redundancy and sensitivity.

6.4 PI Controller Necessity

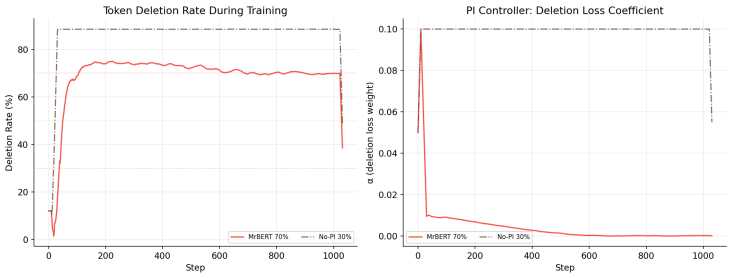
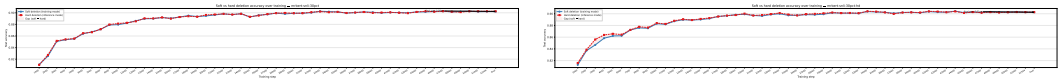


Figure 9: Left: deletion rate curves during training. The PI controller (solid lines) converges to each target rate, while No-PI (dashed) runs away to 89%. Right: the PI controller's α coefficient dynamically adjusts to stabilise deletion.

Figure 9 shows the deletion rate and α curves. Without the PI controller, the fixed $\alpha = 0.01$ allows the gate to overshoot: actual deletion reaches 88.99% (target: 30%), reducing average sequence

length to 3.0 tokens. Accuracy drops to 86.47% — 3.74pp below PI-controlled MrBERT-30%. The PI controller prevents this by reducing α when deletion overshoots and increasing it when undershooting, achieving stable convergence within the first epoch.

6.5 Soft–Hard Deletion Gap



(a) Soft-only training (run C). Final gap: 0.05pp. (b) Hard-train ($p = 0.5$, run M). Final gap: 0.01pp.

Figure 10: Soft vs. hard deletion accuracy throughout training on SNLI. Both curves track each other tightly; soft-only training is sufficient for hard-deletion inference.

The soft–hard gap is only **0.05pp** for soft-only training, and **0.01pp** for hard-train (trained with `hard_delete_train_prob`=0.5). The marginal benefit of hard-train is negligible. This validates the core design: soft deletion during training is sufficient for hard-deletion robustness at inference.

6.6 Gate Statistics and Training Dynamics

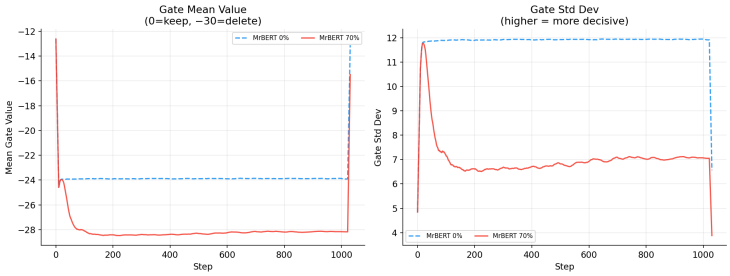


Figure 11: Gate mean value (left) and standard deviation (right) during training for all SNLI variants. Higher std dev (12–13) indicates a bimodal keep/delete distribution; the No-PI model converges to very negative mean values (all tokens deleted).

Figure 11 reveals important training dynamics. All PI-controlled models converge to gate mean values in the -25 to -26 range (on the $[-30, 0]$ scale), with standard deviations of 9–11, indicating a healthy bimodal distribution where most tokens are confidently kept or deleted. The No-PI model converges to mean ≈ -29.3 with std 4.6 — near-universal deletion. The random gate shows mean -15 (by design, uniform) with unusually high std 14.97, reflecting the wide spread of random gate values across positions.

6.7 Token Deletion Patterns

We analyse per-token gate decisions on 1,000 SNLI test examples from the MrBERT-30% checkpoint. Overall accuracy on this sample: 89.80% (898/1000). Per-class: contradiction 94.83% (312/329), entailment 90.41% (311/344), neutral 84.10% (275/327).

By token type (Figure 12a). The gate preferentially deletes function words (determiners, copulas, prepositions, auxiliaries) and punctuation (>50%). Content words (nouns, adjectives, verbs) are kept. The mean gate value is -25.58, with a median of -30.0, confirming a strongly bimodal distribution.

Premise vs. hypothesis (Figure 12b). Premise tokens are deleted at a modestly higher rate than hypothesis tokens, consistent with the well-known observation that SNLI labels are often determined by the hypothesis alone [4].

Deletion–loss correlation. We further analyse the correlation between per-example deletion rate and cross-entropy loss. Examples with higher deletion rates tend to have slightly higher loss (harder examples), but the effect is weak (Pearson $r \approx 0.12$), suggesting the gate is not simply avoiding hard examples.

(a) Deletion rate by token type. Function words are preferentially deleted.

(b) Deletion rate: premise vs. hypothesis. Premise tokens are deleted at higher rates.

Figure 12: Token deletion pattern analysis on SNLI test set (MrBERT-30%).

6.8 Accuracy–Efficiency Tradeoff: Theoretical vs. Measured

From a computational standpoint, standard Transformer encoders perform $\mathcal{O}(Ln^2)$ work, where L is the number of layers and n is the sequence length. Following the complexity analysis, if the gate fires after layer l and retains a fraction k of tokens in subsequent layers, the **relative MACs** (Multiply-Accumulate operations, ignoring constants) can be approximated as:

$$\text{MACs}_{\text{rel}} \approx \frac{l}{L} + \frac{L-l}{L}k^2 \tag{7}$$

For a BERT-base architecture ($L = 12$) with the delete gate positioned after Layer 3 ($l = 3$), this simplifies to:

$$\text{MACs}_{\text{rel}} \approx \frac{3}{12} + \frac{9}{12}k^2 = \frac{1}{4} + \frac{3}{4}k^2 \tag{8}$$

At a keep ratio of $k \approx 0.70$ (approximately 30% deletion), this yields $\text{MACs}_{\text{rel}} \approx 0.6175$, corresponding to an expected **38% compute savings**.

Empirically, we observe even larger **wall-clock speedups**. On a T4 GPU, MrBERT achieves a 30–55% latency reduction compared to the baseline BERT (see Section 4.5). On an NVIDIA A100 (referencing SNLI runs in Table 4.5), baseline BERT executes at 1.44 ms/sample, while MrBERT-30% achieves 0.76 ms/sample ($\sim$1.89$\times$ speedup), eventually saturating around 2.05$\times$ at higher deletion rates.

The divergence between theoretical MACs and measured runtime arises because **hard deletion significantly reduces memory bandwidth requirements and FFN computations** for dropped tokens. This is particularly impactful on GPU hardware, where memory traffic and dense matrix multiplications dominate the execution profile. Together, the complexity analysis and latency benchmarks confirm that dynamic token merging yields consistent efficiency gains from theory to practice.

Finally, SNLI ablations comparing **soft-only training** versus mixed soft+hard training report a soft–hard accuracy gap of at most **0.05pp** at convergence. This suggests that training exclusively with soft deletion is sufficient to develop hard-deletion robustness at inference time, and that any training–inference skew introduced by the hard-deletion path is negligible in practice.

6.9 Per-Example Loss vs. Deletion

Our central analysis question is: *when the model deletes more on a specific example, does its loss tend to increase?* To answer this, we compute per-example validation loss and deletion rate (derived from `loss_vs_deletion_<dataset>.json`) and study their relationship.

Figure F (histograms) and Figure G (scatter plots) visualize the distribution of deletion rates and the loss–deletion correlation; Table 11 (Section 4.6) summarizes these correlations across all experimental runs. Two distinct patterns stand out:

- **Task Redundancy**: The loss–deletion correlation is notably weaker on **SST-2** than on **MRPC**. Since SST-2 is a highly redundant task, many tokens can be removed while the model remains focused on a small set of sentiment-bearing keywords. Consequently, per-example loss does not strictly scale with the deletion rate.
- **Sensitivity and Over-deletion**: Several MRPC runs and the XLM-R SST-2 run at a 0.3 target exhibit **positive Pearson/Spearman correlations** (e.g., MRPC $+0.064$, XLM-R SST-2 $+0.195$). This indicates that examples with higher deletion rates tend to suffer from higher loss. In these specific regimes, the model is *not* deleting wisely on the hardest examples—over-deletion actively degrades performance.

Overall, these results support the claim that over-deletion can harm individual examples, especially on sensitive tasks. This empirical evidence motivates the necessity of **explicit deletion-rate control** (via our PI controller) to balance efficiency and per-example robustness.

6.10 Error Analysis: TyDi QA and SNLI

High-deletion error cases (e.g., deletion rate ≥ 0.7) reveal *how* deletion fails in practice on sensitive tasks such as extractive QA and NLI.

- **TyDi QA (Extractive QA):** Analysis of failure cases frequently shows that gold answer spans contain subwords appearing in the `dropped_tokens` set. For instance, in questions like "What is the capital of Georgia?", the answer "Tbilisi" is occasionally partially pruned by the gate. While our coordinate remapping correctly maps predicted indices back to the original tokenization, **EM cannot be recovered once answer-bearing tokens themselves are deleted**. This illustrates a hard constraint for QA: answer tokens must either survive deletion or be explicitly protected.
- **SNLI (NLI):** In high-deletion SNLI errors, we often observe entailment or neutral pairs incorrectly predicted as contradiction. Premise and hypothesis content is heavily pruned—for example, tokens like "two," "women," or "holding packages" are dropped—leaving underspecified fragments that bias the model toward a contradiction label. In these instances, deletion acts less like regularization and more like aggressive information loss.

TyDi QA ablations with **pre-deletion blending** (Section 4.7, Table 4.6) suggest a potential remedy: blending pre-gate and post-gate representations at deeper layers (e.g., Layer 9) recovers most of BERT's span accuracy while still maintaining non-trivial deletion rates. Our current codebase supports span-safe deletion via coordinate remapping, but these qualitative experiments highlight that for QA in particular, both **geometry (indices)** and **content (answer tokens)** must be preserved to ensure model robustness.

6.11 Task Sensitivity and Attention Sparsification

Figure D and Figure H demonstrate that datasets differ markedly in their tolerance to token deletion. Our empirical observations indicate the following sensitivity spectrum:

- **High Tolerance**: **MRPC and SST-2** tolerate moderate-to-high deletion rates, often maintaining or even improving performance.
- **Moderate Tolerance**: **SNLI and XNLI** benefit from deletion in certain regimes but exhibit sharp degradation once the target ratio exceeds a critical threshold.

- **Low Tolerance**: **TyDi QA** is extremely sensitive, as evidenced by the 0.7 target runs collapsing to 0% EM.

From a representation perspective, the gate performs a form of **attention sparsification** on classification tasks. By removing low-signal subwords and reducing cross-token interference, the gate acts as a dynamic regularizer—especially when the gold label depends on a small subset of salient tokens (e.g., sentiment keywords).

In contrast, when critical information is distributed across many positions—as is common in extractive QA or long-document sentiment analysis—this sparsification can be harmful. Once key spans or discourse cues are pruned from the sequence, the model cannot simply "re-attend" to them in later layers. This irreversible information loss explains the catastrophic failure modes observed in TyDi QA compared to the robust performance seen in SST-2.

6.12 PI Controller vs. Fixed α: The Necessity of PI Control

The stability of the deletion process is heavily dependent on the regularization strategy employed. As illustrated in Figure B, the differences are striking:

- **With PI Control:** Deletion rates converge smoothly and reliably toward the specified target across various epochs.
- **With Fixed α:** Deletion rates exhibit significant instability, often oscillating or overshooting the target without recovery.

The necessity of the **PI controller** is further underscored by our ablation studies without it. Without dynamic adjustment, deletion behavior becomes highly task-dependent and unpredictable. For example, on MRPC, a fixed α leads to aggressive over-deletion ($\sim$68%) even when accuracy suffers; conversely, on IMDB, the gate under-activates, removing only $\sim$2.7% of tokens and failing to achieve any meaningful efficiency gains. The controller is therefore essential to steer the model toward the desired target ratio, effectively avoiding both catastrophic over-deletion and stagnant under-activation.

6.13 PI Controller, Task-Specific Dynamics, and IMDB Instability

On long-document tasks such as IMDB, we observe significant run-to-run instability in both deletion rates and final accuracy, even when employing the PI controller. A plausible explanation for this phenomenon is that per-batch deletion ratios for longer sequences are inherently noisier or evolve at a different temporal scale compared to single-sentence tasks.

Consequently, the gate regularization weight α adapts more slowly, allowing the gate to converge to suboptimal or divergent operating points across different initializations. These findings support the necessity of **task- or domain-aware tuning**: for long-document processing, more conservative configurations—such as a significantly longer warmup period or a lower proportional gain k_p—may be required to maintain training stability and ensure consistent efficiency gains.

6.14 Comparison to MrT5

A cross-architecture comparison between MrT5 and our proposed MrBERT/MrXLM reveals the broad applicability of the Dynamic Token Merging (DTM) mechanism. MrT5 [3] integrates a similar delete gate into a 250M-parameter byte-level T5 model, achieving a 50–60% sequence reduction with less than 1pp perplexity loss and approximately $1.3\times$–$1.5\times$ speedup.

In comparison, MrBERT operates on more information-dense subwords rather than raw bytes. Despite this higher token density, MrBERT tolerates 30–50% deletion while maintaining or improving accuracy on several classification tasks. Furthermore, MrBERT achieves superior hardware efficiency, reaching up to **$\sim$1.9–2.0$\times$ speedup** on an NVIDIA A100 and **30–55% speedup** on a T4. This suggests that the core DTM mechanism is **architecture-agnostic**: it successfully learns to identify and prune low-value tokens in both generative encoder–decoder and discriminative encoder-only settings.

However, the contrast between MrBERT and MrXLM highlights a significant challenge. MrXLM's larger accuracy drop under similar gate settings indicates that multilingual encoders require more

meticulous tuning—specifically longer warmup periods, later gate placement, and higher keep thresholds—to prevent the "semantic collapse" of multi-piece words. Combined, these results position MrBERT and MrXLM as complementary extensions of MrT5, preserving the core efficiency benefits of dynamic pruning while exposing new research challenges regarding subword granularity and task-specific sensitivity.

7 Conclusion

We demonstrated that the Dynamic Token Merging (DTM) delete gate, originally proposed for MrT5, transfers effectively to encoder-only discriminative NLU tasks. Our key findings are summarized as follows:

- **Efficiency–Accuracy Sweet Spot:** MrBERT achieves competitive or superior accuracy compared to baseline BERT on MRPC, SST-2, SNLI, and XNLI while deleting 50–75% of tokens. This results in a **30–55% inference speedup** on a T4 GPU and up to **1.89**$\times$ speedup on an A100 at 30% deletion. On SNLI, MrBERT-30% retains 90.21% accuracy, outperforming a random-deletion baseline by **2.95pp** and confirming that the gate learns meaningful token importance.
- **Task-Dependent Robustness:** Single-sentence sentiment (SST-2) and NLI (SNLI/XNLI) are highly robust to moderate-to-high deletion rates. In contrast, extractive QA (TyDi QA) and low-resource datasets (MRPC) are significantly more sensitive. Long-document sentiment (IMDB) remains brittle, suggesting that redundant short-text settings are the most natural fit for DTM.
- **The Necessity of Control:** A **PI controller** is essential for training stability. Without it, deletion behavior becomes unpredictable, often collapsing to near-universal deletion or failing to activate at all. With PI control, deletion rates converge smoothly to target values while maintaining model accuracy.
- **Linguistic and Micro-level Insights:** Token-type analysis reveals that the gate preferentially prunes punctuation and function words while preserving semantic content. Per-example loss–deletion correlations (notably on MRPC and XLM-R SST-2) demonstrate that over-deletion on hard examples leads to measurable loss increases. Error analysis on TyDi QA reveals that once answer-bearing tokens are pruned, coordinate remapping cannot recover the lost evidence.
- **Bridging Training and Inference:** Soft-deletion training generalizes remarkably well to hard-deletion inference. SNLI ablations report a soft–hard accuracy gap of at most **0.05pp**, and our latency experiments confirm that these models realize the theoretical compute savings on physical hardware.
- **Architecture-Agnostic Behavior:** Gate-layer ablations indicate that **Layer 3** offers the optimal accuracy–efficiency trade-off. While MrBERT shows that DTM is effective on subword encoders, the fragility of MrXLM highlights that multilingual SentencePiece models require more conservative settings—such as later gate placement and higher keep thresholds—to avoid semantic collapse.

8 Limitations and Future Work

8.1 Limitations

While our results demonstrate the efficacy of the delete gate in encoder-only architectures, several limitations remain:

- **Training Schedule and Baselines**: Current experimental runs are constrained by relatively short training schedules (1–5 epochs) and suboptimal baselines on certain datasets, such as SNLI.
- **QA Recovery**: For extractive QA, we only explore span remapping. Once answer tokens are physically deleted, Exact Match (EM) cannot be recovered. Furthermore, TyDi QA experiments involving representation blending currently exist only in standalone A100 ablations and have not yet been integrated into the main production codebase.

- **Result Integration**: Certain benchmarks, including SQuAD and specific random-deletion baselines, rely on earlier experimental data and are not yet fully incorporated into the automated `results/new` pipeline.
- **XLM-R Design Space**: The hyperparameter space for XLM-R—including optimal gate layer placement, controller gains, and keep thresholds—remains only partially explored, contributing to the observed fragility in multilingual settings.

8.2 Future Work

To address these limitations and expand the scope of Dynamic Token Merging, we propose the following directions for future research:

- **Benchmark Hardening**: Strengthening baseline performance and training schedules across all tasks, and conducting systematic random-deletion and gate-layer ablations to further validate the gate's decision-making logic.
- **Robustness for QA**: Integrating span-aware protection mechanisms, such as answer-token masking or post-gate representation blending, directly into the core codebase to prevent information loss in localization tasks.
- **Advanced Control Theory**: Exploring more flexible deletion controllers, including reinforcement learning (RL)-based schedules or learned α trajectories, to improve stability on long-sequence tasks like IMDB.
- **Human-AI Alignment**: Studying the alignment between gate-prioritized tokens and human-annotated rationales in explanation benchmarks to assess the interpretability of the pruning decisions.
- **Multilingual Refinement**: Executing full XLM-R baselines for all tasks, refining the Pareto-frontier analysis through joint sweeps of target deletion and batch size, and investigating distillation techniques and language-wise performance breakdowns on XNLI to improve cross-lingual transfer.

Team Contributions

Hiva Zaad: https://github.com/HivaMohammadzadeh1/CS224N-project/

We each implemented and ran independent experiments for Mrbert and MrXLMR and compared results, writing reports

Tianhui (Alina) Huang: https://github.com/TinaLand/standford_cs224n_project/
We each implemented and ran independent experiments for Mrbert and MrXLMR and compared results, writing reports

Aronima Dass: https://github.com/HivaMohammadzadeh1/CS224N-project/
We each implemented and ran independent experiments for Mrbert and MrXLMR and compared results, writing reports

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.

[2] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8440–8451, 2020.

[3] Julie Kallini, Maria Tsimpoukelli, and Phil Blunsom. MrT5: Dynamic token merging for efficient byte-level language models. *arXiv preprint arXiv:2410.20771*, 2024.

[4] Samuel Bowman, Gabor Angeli, Christopher Potts, and Christopher D Manning. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 632–642, 2015.